

SIMATS ENGINEERING

Co3-ASSESSMENT TOOL 2

Case study

COURSE NAME: Computer Networks

COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO3: *Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

Dr.V.Parthipan

QUESTIONS:4

Total Marks:40

Code of Conduct:

I C Karthik Reddy(192425157) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:C KARTHIK REDDY

Criteria	Understanding the case 2.5	Application of Concepts 2.5	Analysis 2	Solution Design 2	Clarity 1	Total (10)
Q1						
Q2						
Q3						
Q4						

Faculty Signature

QUESTIONS:4

Total Marks:40

1. A multinational IT company uses a cloud-based video conferencing platform for daily client meetings. During peak business hours, employees experience frequent voice distortion, frozen video frames, and delayed communication. Network monitoring reports indicate an **8% packet loss, 180 ms jitter**, and fluctuating bandwidth utilization. The IT administrator suspects that the issue is related to the transport protocol or application-level buffering strategy.

Questions

1. Analyze whether the problem is primarily caused by the **transport protocol (TCP/UDP)** or the **application layer**.
2. Justify your answer based on the communication requirements of real-time multimedia applications.
3. Recommend suitable protocol-level or application-level improvements to enhance conferencing quality.

1.1 Analyze whether the problem is primarily caused by TCP/UDP or the application layer

The problem is primarily related to the transport protocol and application-level buffering strategy, but for real-time video conferencing, UDP is generally more suitable than TCP.

The reported conditions—8% packet loss, 180 ms jitter, and fluctuating bandwidth—are highly problematic for real-time multimedia. If TCP is being used for voice/video, packet retransmissions and congestion control can introduce additional delays. TCP waits for lost packets to be retransmitted, which can cause head-of-line blocking, resulting in frozen video and delayed audio.

However, if UDP is already being used, the application layer must handle packet loss, jitter, and bandwidth variations through techniques such as jitter buffers, adaptive bitrate, packet-loss concealment, and forward error correction.

1.2 Justification based on real-time multimedia requirements

Real-time applications prioritize low latency and continuous playback over perfect delivery.

Requirement	TCP	UDP
Reliability	High	Low
Retransmission	Yes	No

Requirement	TCP	UDP
Latency	Higher	Lower
Head-of-line blocking	Possible	No
Suitable for live audio/video	Less suitable	Highly suitable
Congestion control	Built-in	Application dependent

For conferencing, a slightly lost video packet is generally preferable to waiting hundreds of milliseconds for retransmission. Therefore, UDP with application-level recovery mechanisms is normally preferred.

The 180 ms jitter indicates significant variation in packet arrival time. A jitter buffer can smooth these variations, but an excessively large buffer increases end-to-end delay. The 8% packet loss can cause missing audio/video frames, while fluctuating bandwidth can cause congestion and quality degradation.

1.3 Recommended improvements

Protocol-level improvements:

- Use UDP-based real-time transport for audio/video.
- Use RTP/RTCP for media transport and quality monitoring.
- Apply congestion-control mechanisms suitable for real-time traffic.
- Prioritize voice/video traffic using QoS.
- Use forward error correction where appropriate.

Application-level improvements:

- Implement an adaptive jitter buffer.
- Use adaptive bitrate (ABR) to reduce video quality when bandwidth decreases.
- Apply packet-loss concealment for missing audio/video packets.
- Dynamically adjust video resolution and frame rate.
- Use scalable video coding when available.
- Monitor packet loss, jitter, RTT, and bandwidth continuously.

Conclusion: The best solution is generally UDP + RTP/RTCP + adaptive application-level buffering and bitrate control, rather than relying on TCP retransmissions for real-time media.

2. A multinational software company hosts its web services across multiple continents. Employees frequently report long delays before the application homepage loads. Network analysis shows that the average DNS lookup time is approximately 800 ms, significantly affecting user experience. The organization plans to redesign its DNS infrastructure while ensuring continuous service availability during server failures.

Questions

1. Analyze the reasons behind the high DNS resolution time.
2. Propose a suitable DNS architecture using techniques such as Anycast DNS, DNS prefetching, and TTL optimization to reduce the below 50ms.
3. Evaluate the advantages and limitations of your proposed architecture in terms of performance, scalability, and availability.

A.

2.1 Reasons for high DNS resolution time

An average DNS lookup time of **800 ms** is extremely high for a web application. Possible causes include:

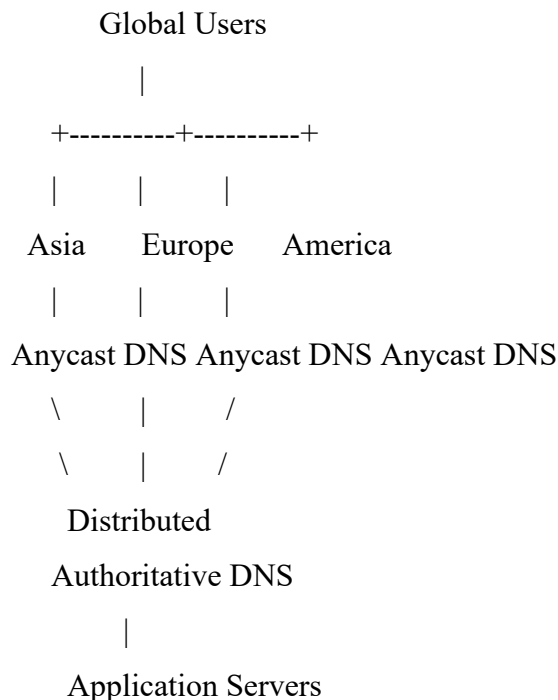
1. **Geographical distance** between users and DNS servers.
2. A centralized DNS infrastructure causing long network round trips.
3. **DNS server overload** during peak periods.
4. Multiple recursive lookups before obtaining the final IP address.
5. Low TTL values causing frequent cache expiration.
6. Poorly configured recursive resolvers.
7. DNS server failures or packet loss causing retries.
8. Lack of geographically distributed DNS infrastructure.

Since DNS resolution occurs before the browser can establish the application connection, an 800 ms lookup can significantly increase page-load time.

2.2 Proposed DNS architecture

A suitable architecture would combine **Anycast DNS, DNS caching/prefetching, optimized TTL values, and geographically distributed authoritative servers.**

Architecture:



Techniques

1. Anycast DNS

The same DNS IP address is advertised from multiple geographically distributed DNS locations. Routing directs users to a nearby DNS node.

This reduces:

- RTT
- Network distance
- DNS response time

2. DNS caching

Frequently requested domain information should be cached by recursive DNS resolvers. Cached responses can be returned without contacting the authoritative server.

3. DNS pre-fetching

Popular or frequently accessed DNS records can be refreshed before their TTL expires, reducing cache-miss delays.

4. TTL optimization

Appropriate TTL values prevent unnecessary DNS queries while still allowing changes to propagate within an acceptable period.

For relatively stable records, a longer TTL can significantly reduce repeated DNS resolution.

5. Multiple authoritative DNS servers

Deploy authoritative servers across different geographical regions and ensure that failure of one server does not interrupt DNS resolution.

2.3 Advantages and limitations

Aspect	Advantages	Limitations
Performance	Lower latency through Anycast and caching	Anycast routing configuration is complex
Scalability	Distributed servers handle large query volumes	More infrastructure is required
Availability	Multiple DNS nodes provide redundancy	Failure detection and routing must be carefully managed
Caching	Reduces repeated DNS queries	Stale information can remain until TTL expires
Pre-fetching	Reduces cache-miss delays	Generates additional DNS traffic
TTL optimization	Improves cache efficiency	Very long TTL delays DNS changes

Expected result: With geographically distributed Anycast DNS, efficient caching, prefetching, and appropriate TTL values, DNS resolution can potentially be reduced from **800 ms to below 50 ms** for users with nearby recursive/authoritative DNS infrastructure. The exact result depends on ISP DNS behavior, routing, network conditions, and cache-hit rates.

3.A cloud service provider delivers applications to millions of users worldwide. During promotional events, users experience intermittent delays in accessing cloud-hosted services. Investigation reveals that DNS queries are taking nearly 800 ms because all requests are directed to a single primary DNS server. The company wants a highly available DNS solution capable of handling global traffic efficiently.

Questions

4. Examine the impact of centralized DNS architecture on application performance.
5. Design a distributed DNS solution that minimizes lookup latency while maintaining high availability.
6. Analyze how TTL values, Anycast routing, and DNS caching influence system performance and fault tolerance.

A.

3.1 Impact of centralized DNS architecture

In the given scenario, all DNS requests are directed to a **single primary DNS server**. This creates a major performance and availability bottleneck.

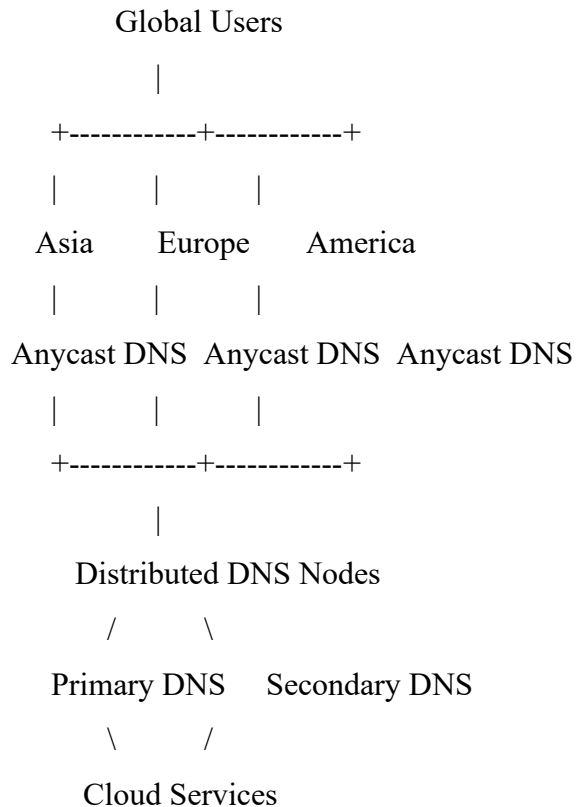
The problems include:

- High query-processing load.
- Increased response time during promotional events.
- Single point of failure.
- Increased queueing and processing delays.
- Greater network distance for geographically distant users.
- Potential DNS server overload.
- Application startup delays because clients must complete DNS resolution first.

If DNS resolution takes **800 ms**, every cache-miss request can experience almost one second of additional delay before connecting to the cloud service.

3.2 Distributed DNS solution

A suitable architecture is:



Main components

1. Anycast DNS

Multiple DNS servers advertise the same IP address. Users are automatically routed to a nearby DNS location.

2. Multiple authoritative servers

Deploy redundant authoritative DNS servers across multiple regions.

3. Recursive caching

DNS resolvers cache frequently requested records, reducing requests to authoritative servers.

4. Load distribution

DNS requests are distributed across multiple nodes instead of being processed by one server.

5. Health monitoring

Failed DNS nodes should be removed from service through routing and health-check mechanisms.

3.3 TTL, Anycast and caching effects

TTL

TTL determines how long a DNS record can remain in a resolver's cache.

- **Higher TTL:** fewer DNS queries, lower latency and DNS load.

- **Lower TTL:** faster propagation of changes but more DNS queries.

Therefore, stable cloud-service records can use relatively longer TTL values, while frequently changing records may require shorter TTLs.

Anycast

Anycast allows multiple DNS servers to use the same service IP address.

Benefits:

- Reduces geographical latency.
- Distributes DNS traffic.
- Improves resilience.
- Automatically directs users toward a nearby network location.

If one Anycast location fails, routing can direct traffic to another available location.

DNS caching

Caching is one of the most important mechanisms for improving DNS performance.

For a cache hit:

Client → Recursive Resolver → Cached IP

The resolver does not need to contact the authoritative DNS server.

For a cache miss:

Client → Recursive Resolver → Root → TLD → Authoritative DNS

The second process requires multiple interactions and therefore takes longer.

Conclusion: A combination of **Anycast + distributed authoritative DNS + DNS caching + optimized TTL values + health monitoring** provides much better performance, scalability, and fault tolerance than a centralized DNS architecture.

4.A financial institution operates an electronic stock trading platform where thousands of buy and sell orders are submitted every second. During market opening hours, the average order acknowledgment latency increases from 1 ms to 40 ms, resulting in delayed trade confirmations and customer dissatisfaction. Network engineers observe increased TCP retransmissions, longer connection establishment times, and excessive buffering.

Questions

1. Analyze three possible TCP-related factors responsible for the latency increase, considering flow control, Nagle's algorithm, and SYN queue limitations.
2. Explain how each factor affects transaction processing in high-frequency systems
3. Recommend appropriate TCP configuration changes to reduce latency and improve transaction reliability.

A.

4.1 Three TCP-related factors

A. TCP Flow Control

TCP uses the **receiver window (rwnd)** to prevent the sender from overwhelming the receiver. If the receiver's buffer becomes full, it advertises a smaller receive window. The sender must then slow down or stop transmitting.

In a high-frequency trading system, this can create:

High order rate



Receiver buffer fills



Smaller TCP receive window



Sender slows transmission



Higher order latency

Excessive buffering can also contribute to increased queueing delay.

B. Nagle's Algorithm

Nagle's algorithm combines small TCP segments into larger segments to improve network efficiency.

However, financial trading applications frequently transmit **small, latency-sensitive messages**.

For example:

BUY 100 AAPL

SELL 50 MSFT

BUY 20 TSLA

If Nagle's algorithm waits for acknowledgment before sending additional small packets, these delays can be significant.

For latency-sensitive transactions, **TCP_NODELAY** can be enabled to disable Nagle's algorithm.

C. SYN Queue Limitations

Before a TCP connection becomes fully established, it follows:

Client → SYN

Server → SYN-ACK

Client → ACK

During heavy market-opening traffic, thousands of connection attempts may arrive simultaneously.

If the server's **SYN backlog/queue** is insufficient:

High connection requests

↓

SYN queue becomes full

↓

Dropped/delayed SYN requests

↓

Retransmissions

↓

Longer connection establishment

↓

Higher transaction latency

This is particularly problematic when the application frequently establishes new TCP connections.

4.2 Effect on high-frequency transaction processing

Factor	Effect on trading system
Flow control	Limits sender transmission when receiver buffers are congested
Nagle's algorithm	Delays small transaction messages
SYN queue limitation	Delays or drops new connection attempts
TCP retransmissions	Increase transaction completion time
Excessive buffering	Creates queueing and latency
Connection establishment	Adds overhead before orders can be transmitted

In high-frequency systems, even a few milliseconds can affect order processing and market responsiveness.

4.3 Recommended TCP configuration changes

1. Disable Nagle's algorithm

Use:

`TCP_NODELAY = enabled`

This allows small trading messages to be transmitted immediately instead of waiting for additional data.

2. Increase SYN backlog

Increase the server's TCP SYN queue/backlog capacity so that it can handle large bursts of connection requests.

3. Optimize receive and send buffers

Configure TCP buffer sizes according to traffic requirements. Avoid unnecessarily large buffers that can increase queueing delay.

4. Use persistent TCP connections

Instead of creating a new TCP connection for every order:

Bad:

Order → TCP connection → Order

Order → TCP connection → Order

Prefer:

TCP connection

↓

Order 1

Order 2

Order 3

Order 4

...

This eliminates repeated three-way handshake overhead.

5. Reduce packet loss

Since retransmissions are already observed, investigate:

- Network congestion
- Interface errors
- Router/switch queues
- Buffer overflow
- Packet drops

Reducing packet loss directly reduces TCP retransmissions and latency.

6. Consider low-latency transport architecture

For extremely latency-sensitive systems, the organization can consider specialized market-data/order protocols and carefully optimized networking infrastructure rather than relying solely on default TCP settings.